

**INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY**

**CAMPUS MONTERREY**



**Diseño de sistemas en chip - TE2003B**  
**Grupo: 603**

**Acciones de mejora en la graficación en RPI**

Roberto Raymundo Gómez Vargas - A01285451

Eugenio Javier Martinez Gastelum - A01721946

Juan Manuel Félix Campos - A00838053

Ivan Rodriguez Nuñez-A01412462

Docentes:

Enrique González Guerrero

Raúl Peña Ortega

Socio formador:

John Deere

# Índice

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>I. Introducción.....</b>                                                        | <b>3</b>  |
| <b>II. Marco Teorico.....</b>                                                      | <b>5</b>  |
| Sistema en Chip (SoC).....                                                         | 5         |
| Microcontrolador.....                                                              | 5         |
| Facilita la Creación de Nuevos Productos.....                                      | 5         |
| Aplicaciones.....                                                                  | 6         |
| Ventajas.....                                                                      | 6         |
| Desventajas.....                                                                   | 6         |
| <b>III. Sistemas Operativos en Tiempo Real (RTOS) para Sistemas Embebidos.....</b> | <b>7</b>  |
| Aplicaciones de RTOS.....                                                          | 7         |
| Ventajas de Usar un RTOS.....                                                      | 8         |
| Desventajas de Usar un RTOS.....                                                   | 8         |
| Ventajas de Usar Bare Metal.....                                                   | 8         |
| Desventajas de Usar Bare Metal.....                                                | 9         |
| <b>IV. Linux Embebido.....</b>                                                     | <b>9</b>  |
| ¿Qué es Linux Embebido?.....                                                       | 9         |
| Aplicaciones de Linux Embebido.....                                                | 9         |
| Ventajas de Linux Embebido.....                                                    | 10        |
| Desventajas de Linux Embebido.....                                                 | 10        |
| Programación con Bash Scripts y Python.....                                        | 11        |
| Ventajas de Programar con Bash y Python.....                                       | 12        |
| Desventajas de Programar con Bash y Python.....                                    | 12        |
| <b>V. Justificación del Problema.....</b>                                          | <b>12</b> |
| <b>VI. Metodologia.....</b>                                                        | <b>13</b> |
| Desarrollado de la problemática.....                                               | 14        |
| Figura 1. conceptualización, prototipado, e implementación.....                    | 14        |
| Hardware Utilizados.....                                                           | 15        |
| Figura 2. conceptualización, prototipado, e implementación.....                    | 15        |
| Software Utilizados.....                                                           | 16        |
| Figura 3. Software Utilizados.....                                                 | 16        |
| Recursos Utilizados.....                                                           | 17        |
| Figura 4. Esquema STM y Raspberry.....                                             | 18        |
| Figura 5. Esquema Teclado y Throttle.....                                          | 19        |
| Figura 6. Esquema Lcd y OLED.....                                                  | 19        |
| Figura 7. Esquema LEDS y Servomotor.....                                           | 20        |

|                                                   |           |
|---------------------------------------------------|-----------|
| Figura 8. Esquema FTDI.....                       | 20        |
| Figura 9. Impresión 3D.....                       | 21        |
| Diagrama esquemático.....                         | 21        |
| Logística e implementación.....                   | 22        |
| Figura 10. Logística.....                         | 22        |
| Diagrama de interacción de las Tareas (RTOS)..... | 22        |
| Figura 11. RTOS.....                              | 23        |
| Diagramas de Flujo por Tarea.....                 | 23        |
| Figura 12. Matrix Keypad.....                     | 24        |
| Figura 13. ADC.....                               | 24        |
| Figura 14. Model step.....                        | 25        |
| Figura 15. UART.....                              | 26        |
| Figura 16. LCD Write.....                         | 28        |
| Scheduling de Tareas (RTOS).....                  | 28        |
| Acciones de mejora gráfico en tiempo real.....    | 31        |
| A. Porque.....                                    | 31        |
| B. Antecedentes.....                              | 32        |
| C. Programa.....                                  | 34        |
| D. Video.....                                     | 44        |
| Interfaz gráfica desarrollada en Python.....      | 44        |
| <b>VII. Conclusiones Individuales.....</b>        | <b>45</b> |
| Roberto Raymundo Gómez Vargas:.....               | 45        |
| Juan Manuel Félix Campos:.....                    | 45        |
| Eugenio Javier Martinez Gastelum:.....            | 46        |
| Ivan Rodriguez Nuñez:.....                        | 46        |
| <b>VIII. Trabajo a Futuro.....</b>                | <b>47</b> |
| Pruebas Extensivas:.....                          | 47        |
| Expansión de Funcionalidades:.....                | 47        |
| <b>IX. Bibliografía.....</b>                      | <b>47</b> |

## **I. Introducción**

La integración de sistemas inteligentes de conducción en tractores es fundamental para modernizar la agricultura y afrontar los crecientes desafíos de la producción de alimentos en un mundo que demanda cada vez más eficiencia y sostenibilidad. Estos sistemas son esenciales porque incrementan la seguridad al reducir los errores humanos y los accidentes en el campo, optimizan el flujo de trabajo y los tiempos de operación, y permiten un uso más eficiente del combustible, lo cual resulta en una disminución significativa de las emisiones contaminantes. Además, facilitan la operación para agricultores con menos experiencia, permitiendo que se concentren en tareas de supervisión y gestión más complejas, lo que mejora la productividad general y la gestión de las explotaciones agrícolas.

Las soluciones actuales para los sistemas de conducción en tractores abarcan desde los Asistentes de Conducción Avanzada , que ofrecen funciones como el frenado automático y el mantenimiento de trayectoria, mejorando la seguridad y el confort del operador, hasta los tractores autónomos, que utilizan inteligencia artificial y una variedad de sensores para realizar tareas sin intervención humana directa, incrementando la eficiencia y la precisión en las labores agrícolas. Además, la conectividad V2X (Vehicle-to-Everything) facilita la comunicación entre tractores y la infraestructura agrícola, mejorando la coordinación y la optimización de las tareas. La navegación predictiva utiliza datos en tiempo real para evitar áreas congestionadas y planificar rutas eficientes, mientras que el platooning permite que varios tractores operen en convoy, reduciendo la resistencia aerodinámica y mejorando la eficiencia operativa y de combustible.

En este contexto, John Deere nos plantea un reto significativo como Socio Formador: diseñar un sistema inteligente de conducción para un tractor agrícola. Este reto nos permitirá vivir todas las etapas del desarrollo, desde la planeación hasta la implementación y la validación de un prototipo funcional, utilizando tarjetas de desarrollo STM32 y Raspberry Pi. Bajo la supervisión de expertos de John Deere, adquiriremos y aplicaremos competencias clave en la creación de soluciones innovadoras para la agricultura de precisión. Este proyecto no solo mejorará la eficiencia y la sostenibilidad de las operaciones agrícolas, sino que también nos brindará una experiencia práctica invaluable en el desarrollo de tecnologías avanzadas para el futuro de la agricultura.

## **II. Marco Teórico**

### Sistema en Chip (SoC)

Un Sistema en Chip (SoC) es un circuito integrado que integra todos los componentes de un sistema electrónico en un solo chip. Esto incluye procesadores, memoria, interfaces de comunicación y periféricos. Los SoCs están diseñados para realizar funciones complejas y son utilizados en una amplia variedad de dispositivos, desde teléfonos móviles y tablets hasta dispositivos de Internet de las Cosas (IoT) y sistemas embebidos.

### Microcontrolador

Un microcontrolador es un tipo de SoC que incluye un procesador, memoria (RAM y ROM) y periféricos de entrada/salida en un solo chip. Los microcontroladores están diseñados para tareas específicas y se utilizan en aplicaciones donde se necesita un control preciso de las funciones del dispositivo. Ejemplos comunes de microcontroladores incluyen los utilizados en electrodomésticos, automóviles, equipos médicos y dispositivos industriales.

## Facilita la Creación de Nuevos Productos

La integración de múltiples componentes en un solo chip simplifica el diseño y la fabricación de dispositivos electrónicos. Esto reduce el tamaño, el costo y el consumo de energía de los productos, permitiendo una mayor miniaturización y eficiencia. Además, los SoCs y microcontroladores proporcionan una plataforma versátil y programable que facilita la innovación y el desarrollo rápido de nuevas funciones y productos.

## Aplicaciones

- SoCs: Utilizados en smartphones, tablets, dispositivos de IoT, sistemas de entretenimiento en automóviles, y más.
- Microcontroladores: Empleados en electrodomésticos, sistemas de control industrial, dispositivos médicos, automóviles, juguetes electrónicos, y sistemas de automatización del hogar.

## Ventajas

- Integración: Combina múltiples componentes en un solo chip, reduciendo el tamaño y el costo.
- Eficiencia Energética: Menor consumo de energía debido a la proximidad de los componentes.
- Velocidad: Mayor velocidad de procesamiento gracias a la integración estrecha de componentes.
- Facilidad de Desarrollo: Proporciona plataformas versátiles y programables para un desarrollo rápido y eficiente de nuevos productos.

## Desventajas

- Complejidad de Diseño: El diseño de SoCs puede ser complejo y requerir conocimientos especializados.

- Costo de Desarrollo: La investigación y el desarrollo inicial pueden ser costosos debido a la necesidad de herramientas y tecnologías avanzadas.
- Flexibilidad Limitada: Una vez fabricado, el hardware de un SoC es difícil de modificar, lo que puede limitar la flexibilidad en caso de cambios necesarios o actualizaciones.

### **III. Sistemas Operativos en Tiempo Real (RTOS) para Sistemas Embebidos**

#### ¿Qué es un RTOS?

Un Sistema Operativo en Tiempo Real (RTOS, por sus siglas en inglés) es un sistema operativo diseñado para aplicaciones que requieren un alto grado de predictibilidad y cumplimiento de plazos específicos. Un RTOS gestiona los recursos del sistema de manera que las tareas críticas se ejecuten en el tiempo previsto, garantizando respuestas rápidas y deterministas a eventos externos.

#### Aplicaciones de RTOS

1. Automoción: Control de motores, sistemas de frenos ABS, y sistemas de asistencia al conductor.
2. Aeroespacial: Sistemas de control de vuelo, navegación y comunicaciones.
3. Medicina: Dispositivos de monitoreo de pacientes, equipos de diagnóstico y sistemas de administración de medicamentos.
4. Telecomunicaciones: Enrutadores, switches y sistemas de comunicación de alta velocidad.
5. Industrial: Controladores lógicos programables (PLC), sistemas de automatización y robótica.
6. Electrodomésticos: Lavadoras, refrigeradores y otros dispositivos inteligentes.

#### Ventajas de Usar un RTOS

1. Determinismo: Garantiza la ejecución de tareas en plazos específicos, crucial para aplicaciones críticas.

2. Gestión de Tareas: Facilita la gestión y priorización de múltiples tareas, mejorando la eficiencia del sistema.
3. Modularidad: Permite desarrollar aplicaciones de manera modular, facilitando el mantenimiento y la actualización.
4. Multitarea: Soporta la ejecución concurrente de múltiples tareas, mejorando la utilización de recursos.
5. Seguridad y Confiabilidad: Proporciona mecanismos robustos de manejo de errores y recuperación.

#### Desventajas de Usar un RTOS

1. Complejidad: Mayor complejidad en el diseño y desarrollo en comparación con aplicaciones bare metal.
2. Requerimientos de Recursos: Consume más memoria y recursos del procesador debido a las funciones adicionales del sistema operativo.
3. Costo: Puede incrementar los costos de desarrollo y licenciamiento.

#### Ventajas de Usar Bare Metal

1. Simplicidad: Menor complejidad en el diseño y desarrollo, más fácil de entender y mantener.
2. Eficiencia: Menor sobrecarga de recursos, lo que puede resultar en un rendimiento más alto para aplicaciones específicas.
3. Costo: Puede ser más económico en términos de desarrollo y recursos de hardware.

#### Desventajas de Usar Bare Metal

1. Falta de Multitarea: Sin soporte nativo para multitarea, lo que complica la gestión de múltiples funciones simultáneas.
2. Escalabilidad: Menos flexible y escalable en comparación con un sistema operativo.

3. Desarrollo y Mantenimiento: Mayor esfuerzo en el desarrollo y mantenimiento, ya que todo debe ser gestionado manualmente.

#### **IV. Linux Embebido**

##### ¿Qué es Linux Embebido?

Linux embebido es una versión del sistema operativo Linux diseñada para ser utilizada en dispositivos embebidos. Estos dispositivos suelen tener recursos limitados, como menos memoria y potencia de procesamiento en comparación con las computadoras de escritorio. Linux embebido se adapta a estas restricciones proporcionando una versión ligera y personalizada del sistema operativo que puede ejecutarse en una amplia variedad de hardware embebido.

##### Aplicaciones de Linux Embebido

1. Dispositivos IoT (Internet de las Cosas): Sensores inteligentes, cámaras de seguridad, y dispositivos domésticos conectados.
2. Automoción: Sistemas de infoentretenimiento, sistemas de navegación y controladores de motor.
3. Telecomunicaciones: Routers, switches y otros equipos de red.
4. Electrodomésticos: Televisores inteligentes, refrigeradores y lavadoras conectadas.
5. Medicina: Equipos de monitoreo de pacientes, dispositivos de diagnóstico y sistemas de administración de medicamentos.
6. Industrial: Controladores lógicos programables (PLC), sistemas de automatización y robótica.

### Ventajas de Linux Embebido

1. Open Source: Código fuente abierto y personalizable, lo que permite ajustes específicos según las necesidades del proyecto.
2. Comunidad y Soporte: Gran comunidad de desarrolladores y abundancia de recursos de soporte, documentación y bibliotecas.
3. Flexibilidad: Soporta una amplia variedad de hardware y puede adaptarse a diferentes requisitos de aplicaciones.
4. Seguridad: Acceso a herramientas y parches de seguridad actualizados, esenciales para aplicaciones críticas.
5. Coste: No hay costos de licenciamiento, lo que puede reducir significativamente los costos del proyecto.

### Desventajas de Linux Embebido

1. Curva de Aprendizaje: Puede ser complejo para desarrolladores novatos debido a su flexibilidad y cantidad de opciones de configuración.
2. Tamaño: Aunque se puede optimizar, Linux embebido puede ser más grande en comparación con otros sistemas operativos embebidos más simples.
3. Rendimiento: La sobrecarga del sistema operativo puede no ser ideal para aplicaciones extremadamente críticas en tiempo real sin personalización adicional.
4. Requisitos de Recursos: Requiere más recursos en términos de memoria y almacenamiento en comparación con sistemas embebidos más ligeros.

## Programación con Bash Scripts y Python

Los Bash scripts son útiles para automatizar tareas de administración del sistema en Linux embebido. Pueden ser utilizados para:

1. Automatización de Tareas: Ejecutar secuencias de comandos para realizar tareas repetitivas.
2. Gestión de Archivos: Manipular archivos y directorios, copiar, mover, eliminar, etc.
3. Configuración del Sistema: Configurar parámetros del sistema y servicios durante el arranque.
4. Monitorización: Supervisar el rendimiento del sistema y recursos.

## Python

Python es un lenguaje de programación versátil y potente que es ampliamente utilizado en sistemas embebidos debido a su simplicidad y amplio ecosistema de bibliotecas. Aplicaciones de Python en Linux embebido incluyen:

1. Desarrollo de Aplicaciones: Creación de aplicaciones personalizadas para control y monitoreo de dispositivos embebidos.
2. Procesamiento de Datos: Manejo y análisis de datos recogidos de sensores y otros dispositivos.
3. Automatización: Automatizar tareas complejas de administración del sistema y control de hardware.
4. Interfaz de Usuario: Desarrollo de interfaces de usuario simples y efectivas para interactuar con el sistema embebido.

## Ventajas de Programar con Bash y Python

- Bash: Ideal para tareas rápidas y scripts de administración del sistema.
- Python: Proporciona una sintaxis simple y limpia, amplia gama de bibliotecas y soporte para el desarrollo rápido de aplicaciones.

## Desventajas de Programar con Bash y Python

- Bash: Puede ser limitado en términos de complejidad y mantenimiento de scripts grandes.
- Python: Aunque versátil, puede ser más lento en comparación con lenguajes compilados para ciertas tareas críticas de rendimiento.

## **V. Justificación del Problema**

En el contexto de la agricultura moderna, la integración de sistemas inteligentes de conducción en tractores se ha vuelto esencial para aumentar la eficiencia y la precisión en las operaciones agrícolas. John Deere, como Socio Formador, ha identificado la necesidad de desarrollar un sistema avanzado que pueda enfrentar los desafíos actuales del sector. Estos desafíos incluyen la necesidad de reducir los errores humanos, mejorar la seguridad y optimizar el uso de recursos en un entorno de trabajo cada vez más exigente.

Actualmente, se estima que alrededor del 40% del valor de producción de un tractor agrícola proviene de los sistemas electrónicos que incorpora. Estos sistemas incluyen microcontroladores y microprocesadores que gestionan funciones críticas como el control de aceleración, frenos, inyección de combustible y sistemas de navegación. Con la evolución hacia vehículos autónomos, estos sistemas se vuelven aún más complejos y esenciales para el funcionamiento eficiente y seguro de los tractores.

La problemática planteada por John Deere se centra en el desarrollo de un sistema inteligente de conducción que no solo mejore la precisión del sembrado y otras tareas agrícolas, sino que también incremente la seguridad y reduzca la fatiga del operador. Este sistema debe ser capaz de integrarse con tecnologías actuales y futuras, ofreciendo una plataforma robusta y escalable para la agricultura de precisión.

El reto consiste en diseñar, implementar y validar un prototipo funcional utilizando herramientas como las tarjetas de desarrollo STM32 y Raspberry Pi. Este desarrollo debe considerar todas las etapas desde la planificación inicial hasta la validación final en un entorno real. La experiencia adquirida en este proceso no solo contribuirá al avance tecnológico en el campo agrícola, sino que también permitirá a los participantes desarrollar competencias clave en el diseño y desarrollo de sistemas embebidos y autónomos.

## **VI. Metodología**

En la fase de conceptualización del proyecto, comenzamos con una visualización gráfica en Python para analizar cómo se comportaría el tractor, enfocándonos en el comportamiento del RPM. Esta visualización inicial nos proporcionó una comprensión más profunda y detallada del funcionamiento del tractor, permitiéndonos identificar y prever posibles desafíos y comportamientos en el sistema. Durante la fase de prototipado, llevamos a cabo la interacción mediante UART entre la Raspberry Pi y la STM32, acompañada de su representación gráfica. Este proceso nos permitió desarrollar un modelo preciso de la transmisión del tractor, así como la gestión y almacenamiento de datos relevantes, y su posterior visualización. En la fase de implementación, avanzamos con el desarrollo completo del sistema utilizando la STM32 y la Raspberry Pi. Implementamos una visualización en tiempo real de los datos, junto con una interfaz gráfica intuitiva que nos permitió manipular diversos parámetros del comportamiento del tractor. Esta interfaz facilitó un control más eficiente y preciso del tractor. Adicionalmente, como parte de la implementación, fabricamos un prototipo del tractor utilizando impresión 3D, lo que nos permitió tener una representación física y tangible del diseño, facilitando así pruebas y ajustes finales del sistema.

### Desarrollado de la problemática

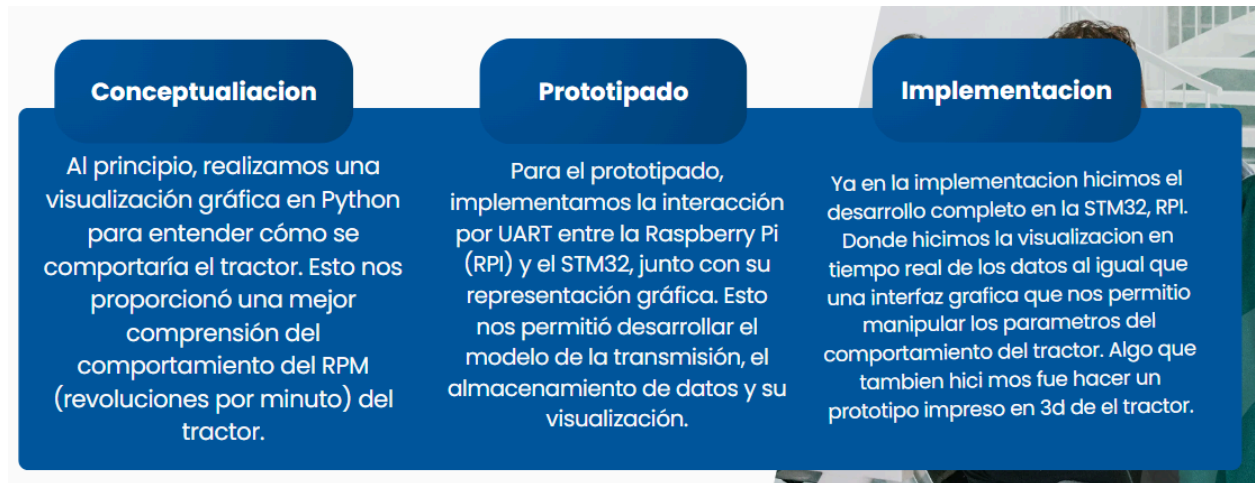


Figura 1. Conceptualización, prototipado, e implementación

### Hardware Utilizados

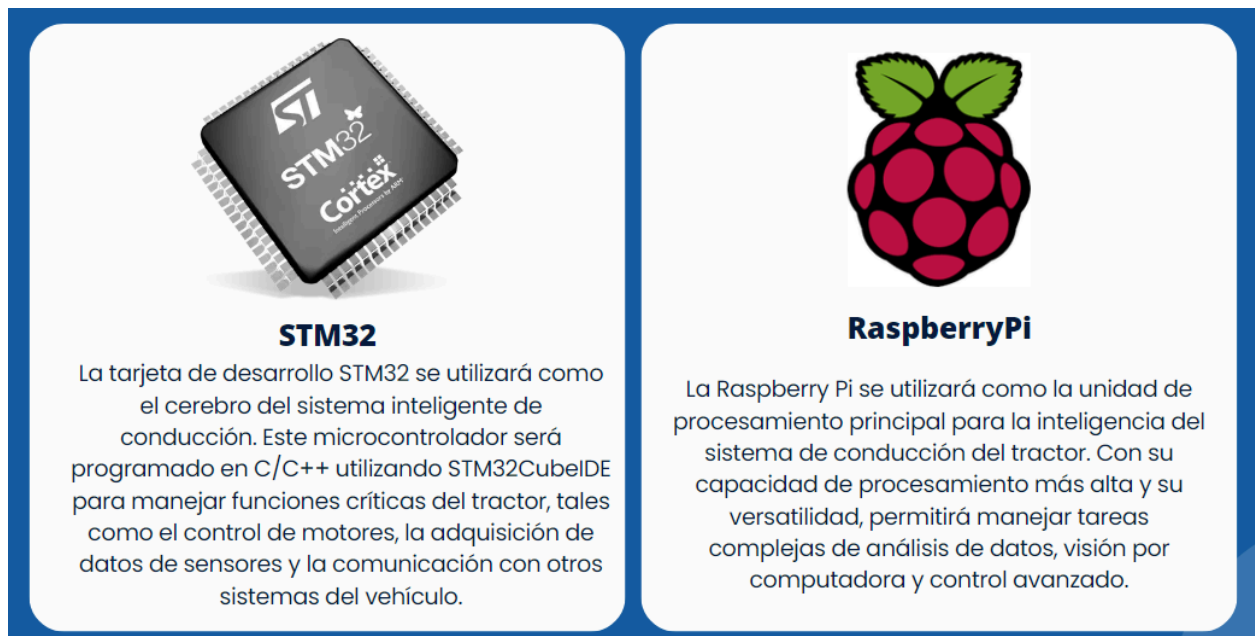
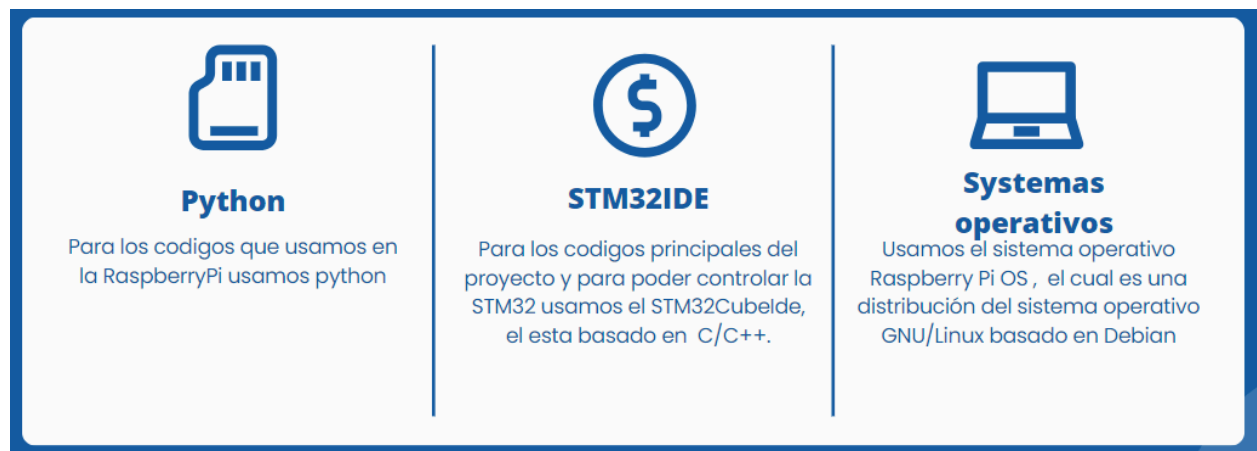


Figura 2. Hardware Utilizados

La tarjeta de desarrollo STM32 se utilizará como el cerebro del sistema inteligente de conducción del tractor. Este microcontrolador, programado en C/C++ utilizando

STM32CubeIDE, manejará funciones críticas del tractor como el control de motores, la adquisición de datos de sensores y la comunicación con otros sistemas del vehículo. Por otro lado, la Raspberry Pi se utilizará como la unidad de procesamiento principal para la inteligencia del sistema de conducción. Gracias a su mayor capacidad de procesamiento y versatilidad, la Raspberry Pi permitirá manejar tareas complejas de análisis de datos, visión por computadora y control avanzado, complementando así las funciones críticas gestionadas por la STM32.

### Software Utilizados



*Figura 3. Software Utilizados*

En nuestro proyecto, empleamos una combinación de lenguajes de programación y entornos de desarrollo para maximizar la eficiencia y funcionalidad del sistema de conducción inteligente del tractor. Para los códigos que corren en la Raspberry Pi, utilizamos Python. Este lenguaje es conocido por su simplicidad y versatilidad, permitiéndonos desarrollar rápidamente scripts para la adquisición y procesamiento de datos, así como para tareas de control avanzado y visión por computadora.

Para los códigos principales del proyecto y el control del microcontrolador STM32, utilizamos el entorno de desarrollo STM32CubeIDE, que está basado en C/C++. Este entorno nos permite escribir y depurar código de manera eficiente, asegurando que el STM32 pueda

manejar funciones críticas del tractor, como el control de motores, la adquisición de datos de sensores y la comunicación con otros sistemas del vehículo.

Además, el sistema operativo elegido para la Raspberry Pi es Raspberry Pi OS, una distribución del sistema operativo GNU/Linux basada en Debian. Esta elección se debe a la estabilidad y flexibilidad que ofrece, junto con un amplio soporte de la comunidad. Raspberry Pi OS nos permite aprovechar al máximo las capacidades de hardware de la Raspberry Pi, facilitando la integración de diferentes módulos y la ejecución de aplicaciones complejas de manera fluida.

En conjunto, el uso de Python, STM32CubeIDE y Raspberry Pi OS proporciona una plataforma robusta y versátil que soporta el desarrollo de un sistema de conducción inteligente eficiente y confiable para el tractor. Esta combinación de tecnologías permite una rápida iteración en el desarrollo, pruebas efectivas y la implementación de características avanzadas, asegurando que el sistema final cumpla con todos los requisitos funcionales y operativos del proyecto.

## Recursos Utilizados

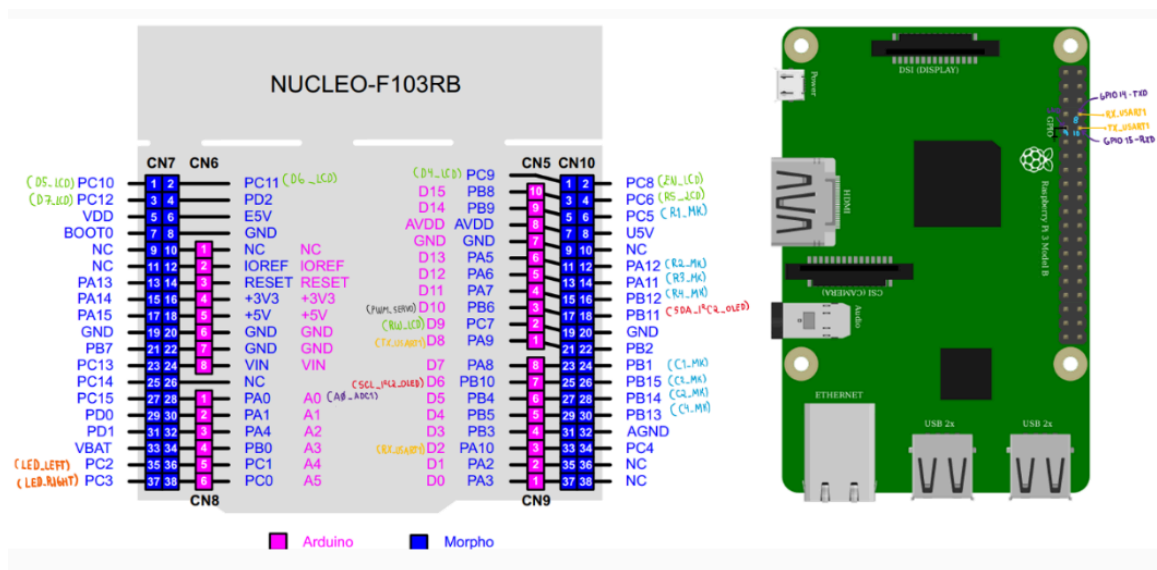


Figura 4. Esquema STM32 y Raspberry

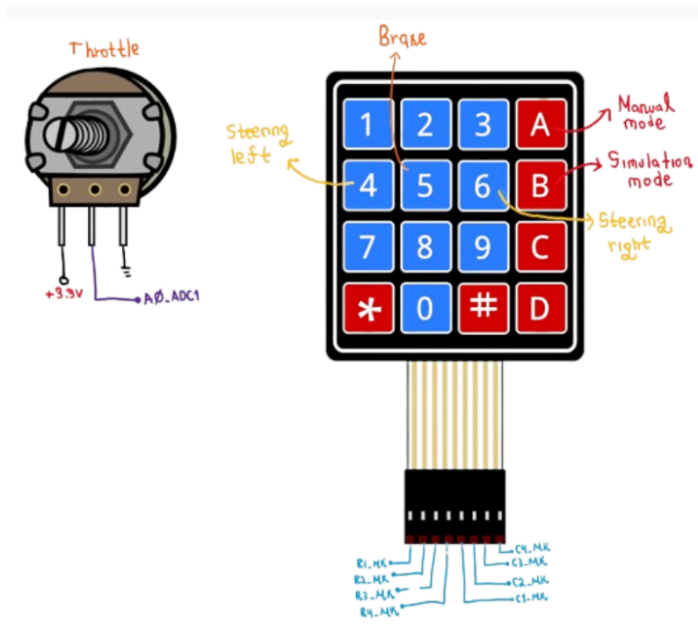


Figura 5. Esquema Teclado y Throttle

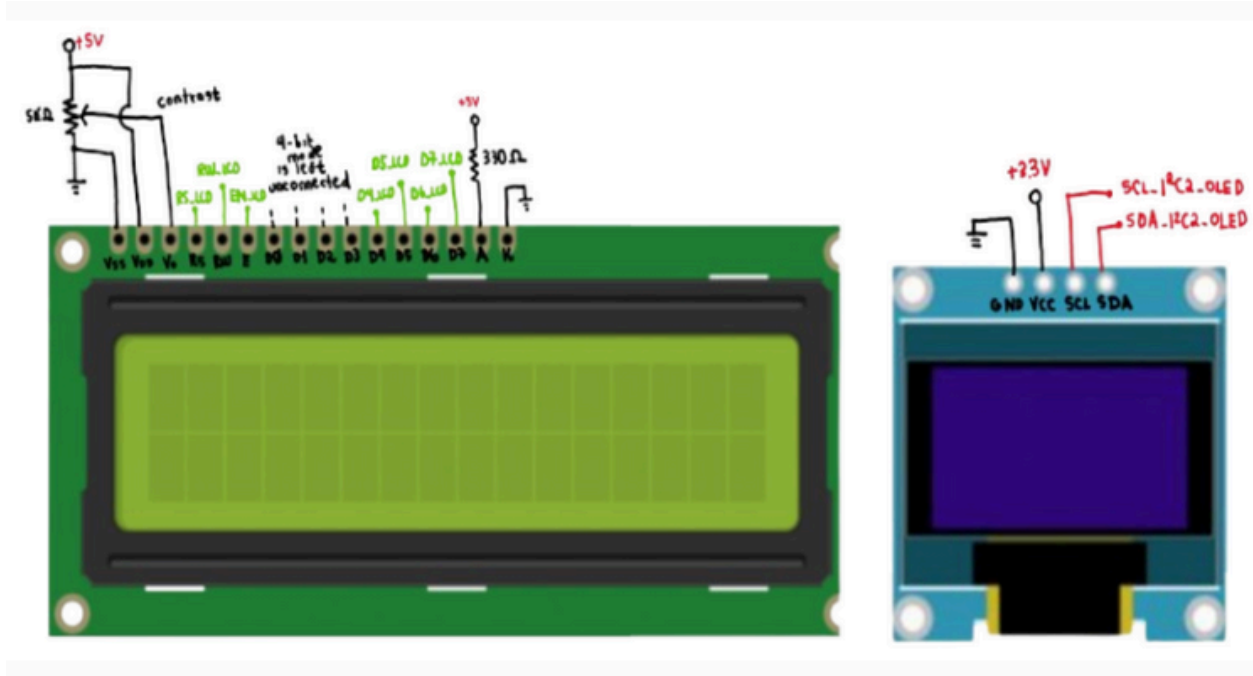


Figura 6. Esquema LCD y OLED

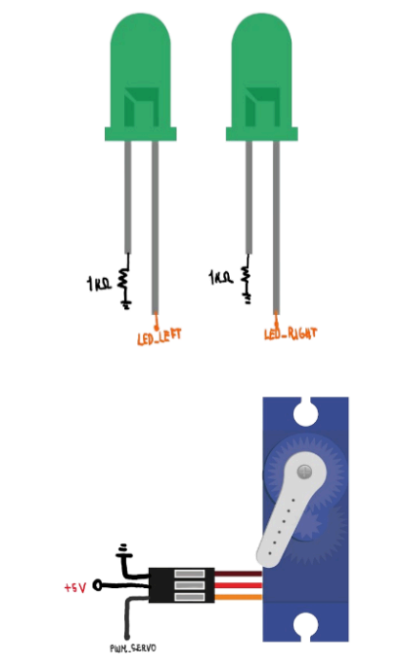


Figura 7. Esquema LEDs y servomotor

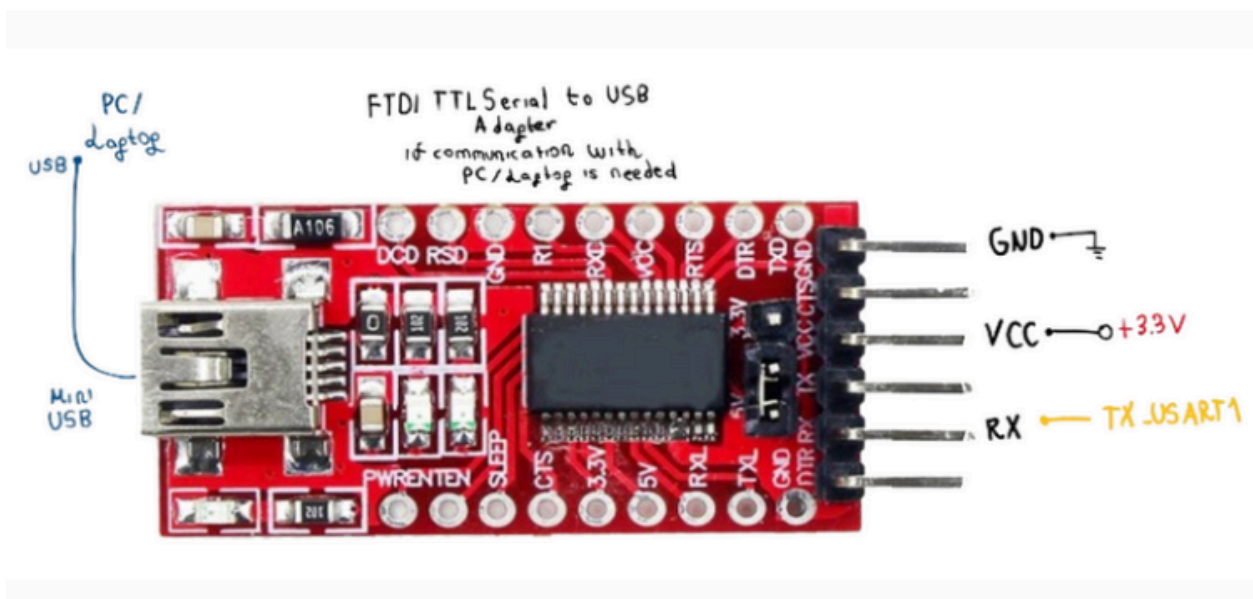
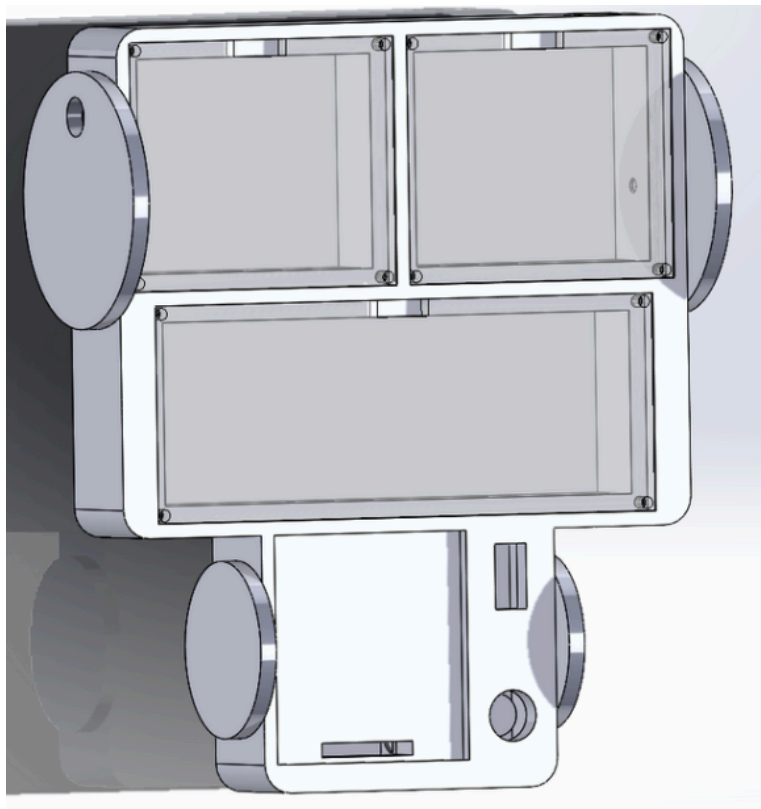


Figura 8. Esquema FTDI



*Figura 9. Impresión 3D*



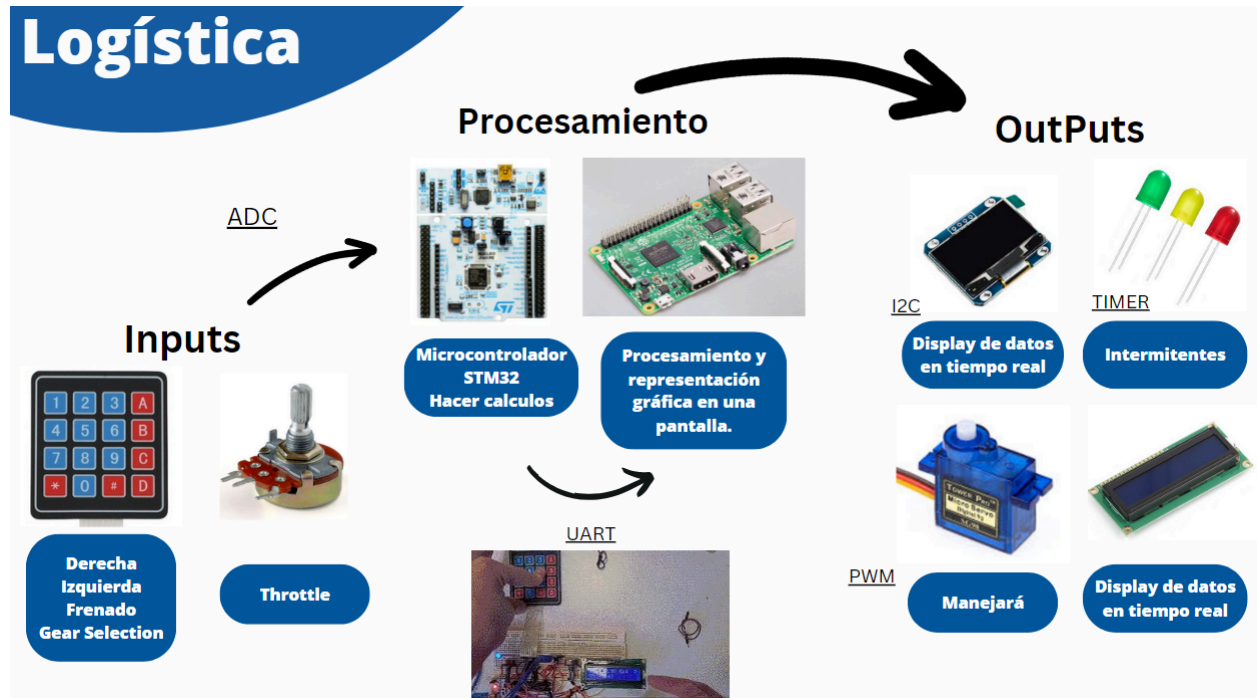
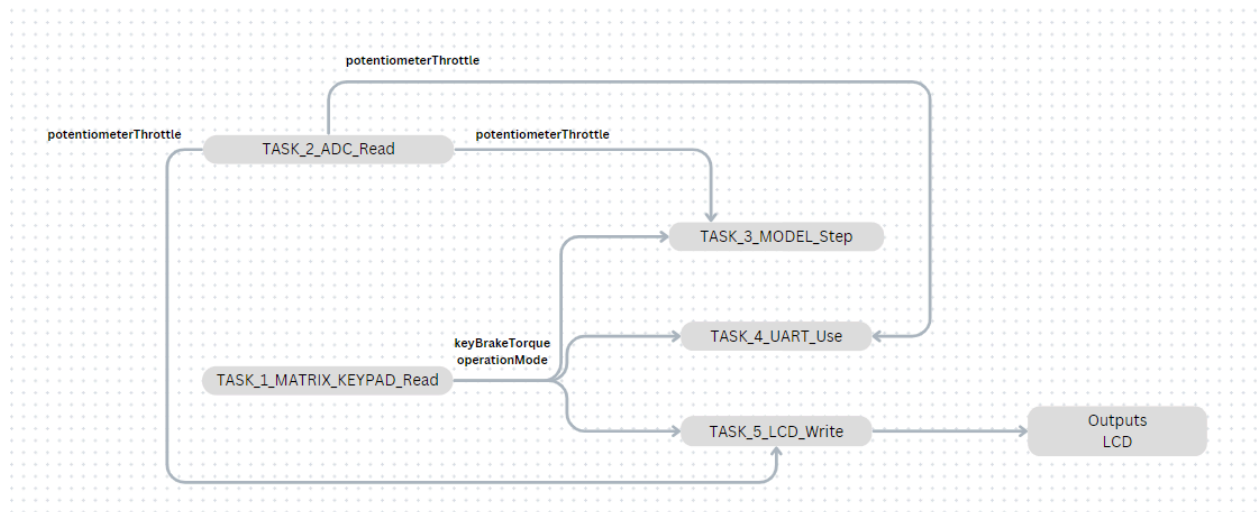


Figura 11. Diagrama de lógica aplicada

## Diagrama de interacción de las Tareas (RTOS)



*Figura 12. RTOS Tareas*

## Diagramas de Flujo por Tarea

### A. Tarea 1; TASK\_1\_MATRIX\_KEYPAD\_Read

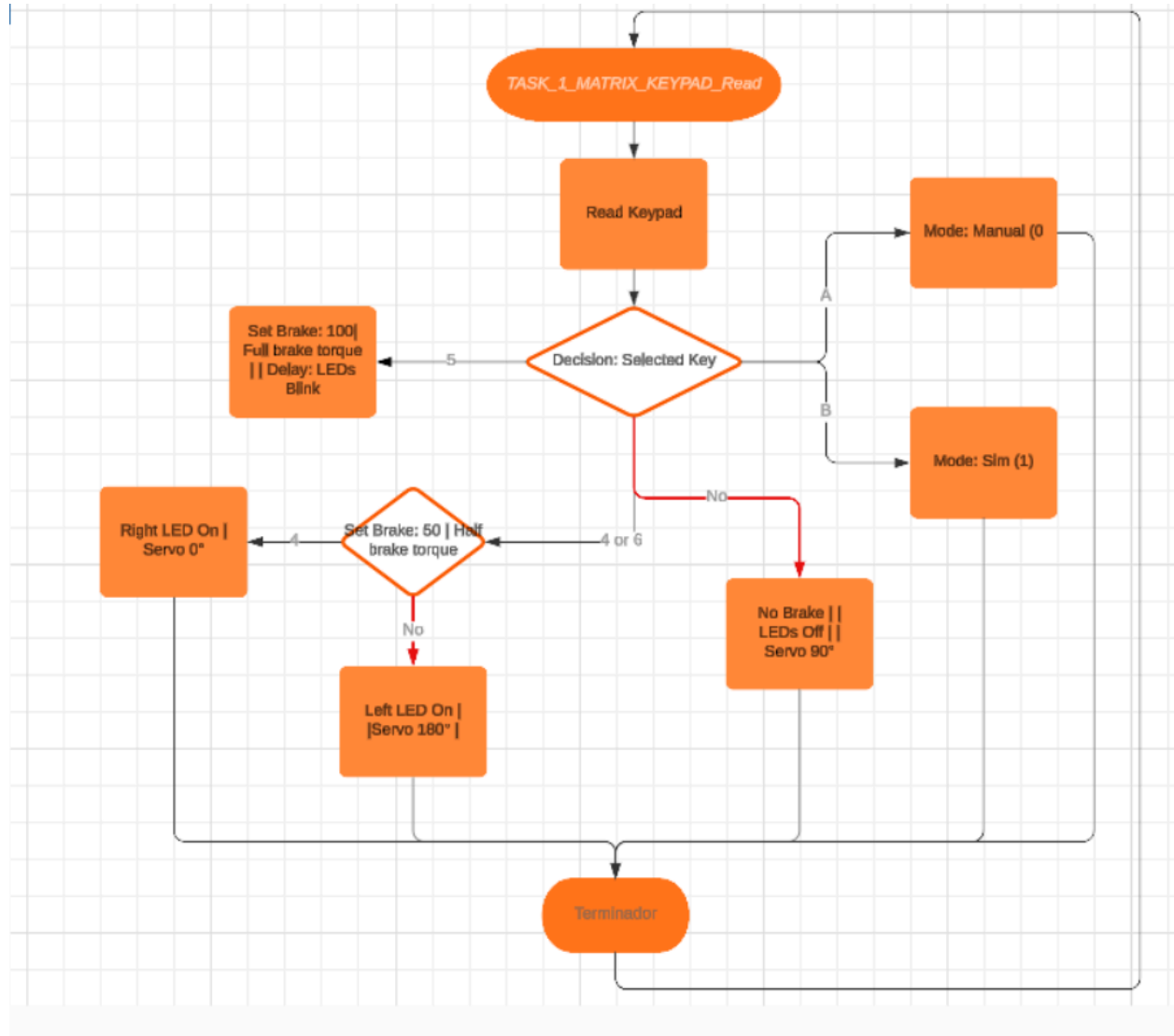


Figura 13. Matrix Keypad

## B. TASK\_2\_ADC\_Read

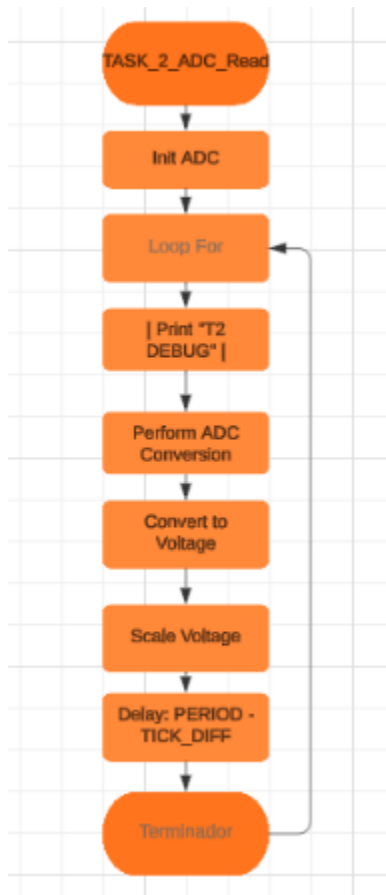


Figura 14. ADC

### C. TASK\_3\_MODEL\_Step

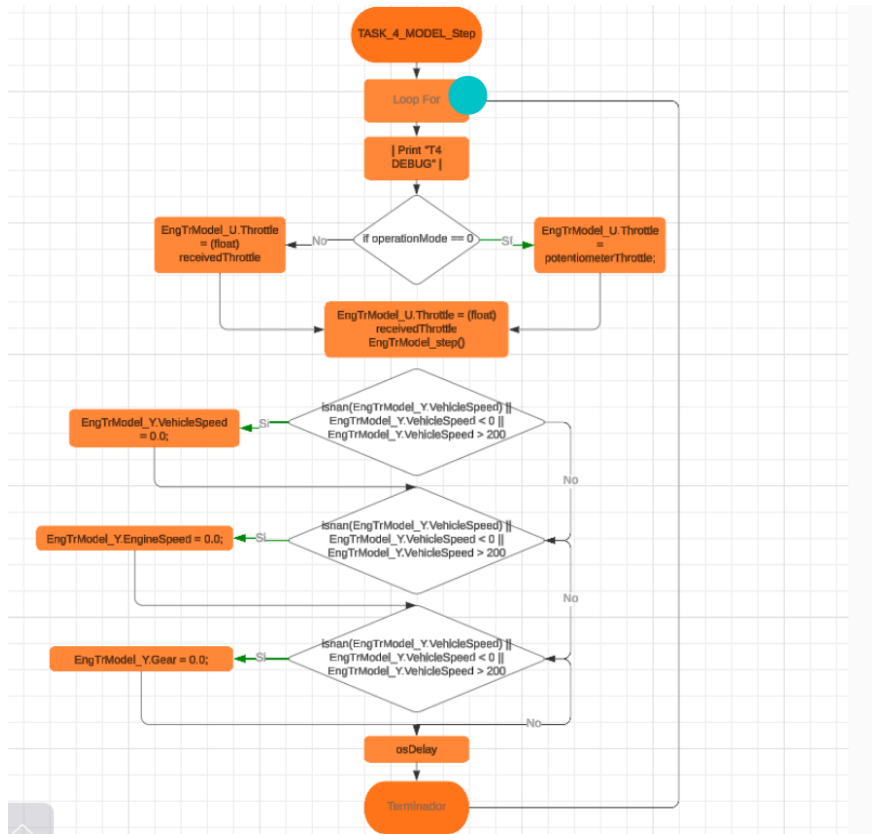
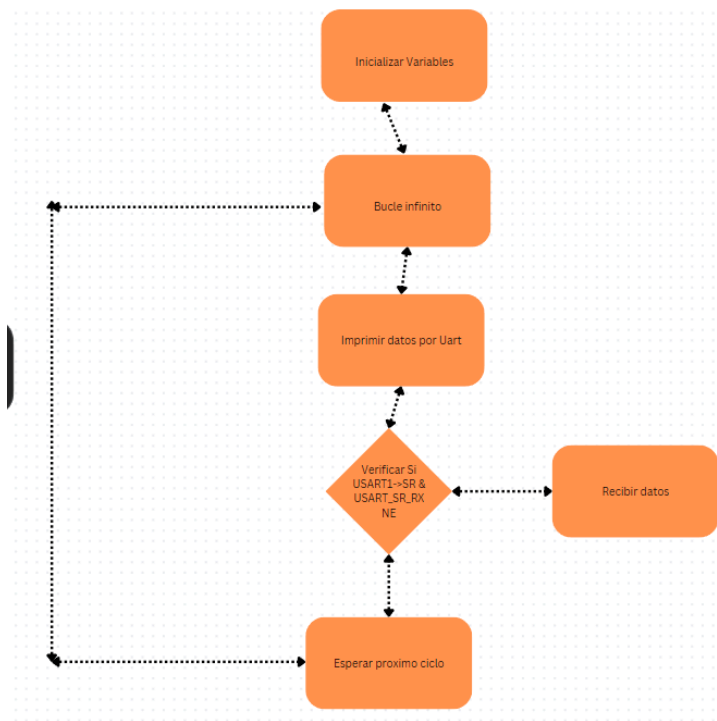


Figura 15 Model step

#### D. TASK\_4\_UART\_Use



*Figura 16. UART*

## E. TASK\_5\_LCD\_Write

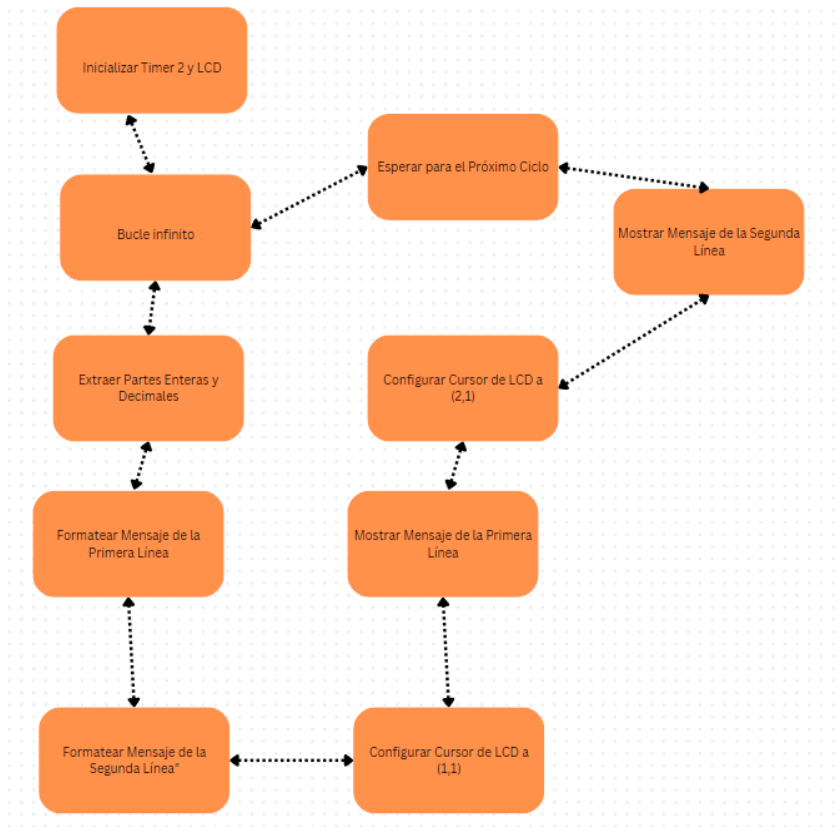


Figura 17. LCD Write

### Scheduling de Tareas (RTOS)

| Process | Period (ms) | Name                      | Execution time (ms) | Priority |
|---------|-------------|---------------------------|---------------------|----------|
| Task1   | 2           | TASK_1_MATRIX_KEYPAD_Read | 0.001               | 1        |
| Task2   | 8           | TASK_2_ADC_Read           | 0.004               | 2        |
| Task3   | 10          | TASK_3_MODEL_Step         | 0.02                | 3        |
| Task4   | 12          | TASK_4_UART_Use           | 2.501               | 4        |
| Task5   | 60          | TASK_5_LCD_Write          | 20                  | 5        |

*Figura 18. Tabla de tareas*

```
osThreadDef(Task2, TASK_2_ADC_Read, osPriorityHigh, 0, 128);  
Task2Handle = osThreadCreate(osThread(Task2), NULL);  
  
osThreadDef(Task4, TASK_4_UART_Use, osPriorityNormal, 0, 256);  
Task4Handle = osThreadCreate(osThread(Task4), NULL);  
  
osThreadDef(Task5, TASK_5_LCD_Write, osPriorityBelowNormal, 0, 128);  
Task5Handle = osThreadCreate(osThread(Task5), NULL);  
  
osThreadDef(Task3, TASK_3_MODEL_Step, osPriorityAboveNormal, 0, 128);  
Task3Handle = osThreadCreate(osThread(Task3), NULL);  
  
osThreadDef(Task1, TASK_1_MATRIX_KEYPAD_Read, osPriorityRealtime, 0, 128);  
Task1Handle = osThreadCreate(osThread(Task1), NULL);
```

*Figura 19. Tareas implementadas en código*

# Schedule Visualizer

Visualizations for Rate-Monotonic Scheduling (RMS) and Earliest Deadline First (EDF) scheduling algorithms.

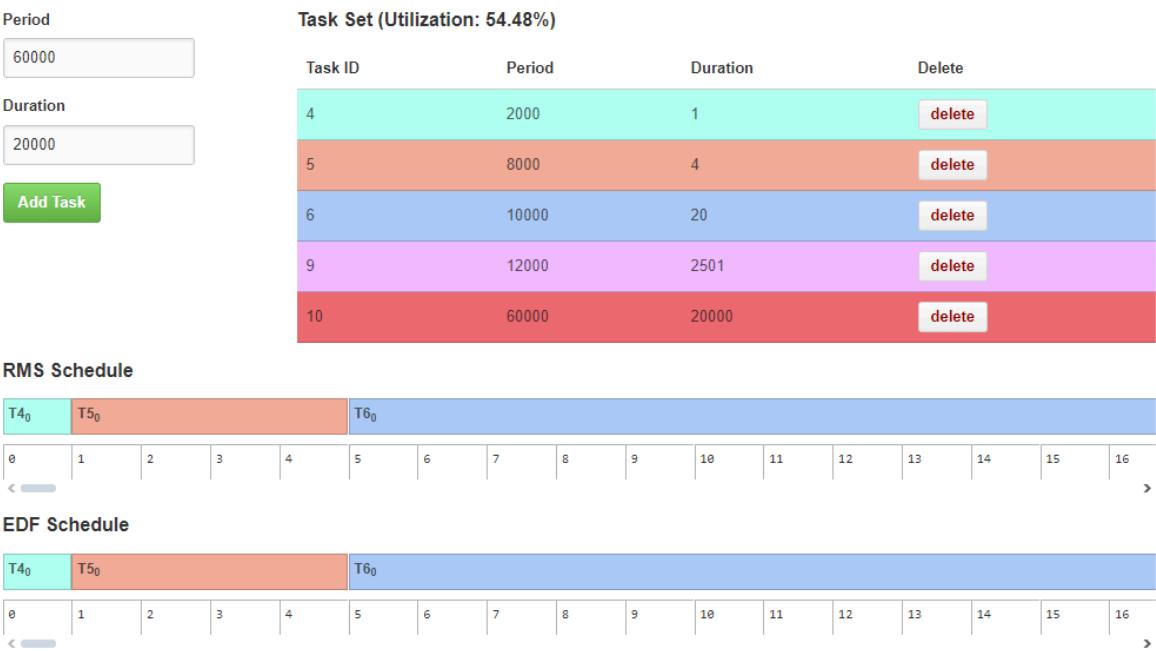


Figura 20. Tareas implementadas en plataforma RMA

## Acciones de mejora gráfico en tiempo real

### A. Justificación

En nuestro proyecto, el monitoreo en tiempo real es esencial para garantizar la seguridad y la eficiencia en la operación de sistemas críticos, como el seguimiento de parámetros en vehículos. La capacidad de obtener datos instantáneos y precisos permite una detección inmediata de situaciones potencialmente peligrosas, como cambios bruscos en la velocidad del motor o problemas en la transmisión. Esta capacidad de respuesta rápida es crucial para prevenir accidentes graves y tomar decisiones informadas en un entorno dinámico.

Además de la seguridad, el monitoreo en tiempo real mejora la experiencia del usuario al proporcionar una interfaz más fluida y responsiva. La visualización instantánea de datos no solo permite una toma de decisiones más efectiva, sino que también contribuye a la eficiencia operativa al identificar rápidamente problemas de rendimiento y garantizar el cumplimiento de normas y regulaciones.

“The use of data analytics in agriculture has the power to revolutionize the industry, driving innovation, boosting productivity, and strengthening food security. Through data analysis, farmers can access valuable information on soil quality, weather forecasts, and crop health, allowing for precise farming methods that optimize resources. This advancement leads to higher crop yields and more efficient management of water, fertilizers, and pesticides. Data analytics not only aids farmers in making well-informed decisions but also fosters a dynamic environment where technology and continuous improvement are key. This transformation improves agriculture's ability to withstand climate change, minimizes environmental harm, and promotes sustainable practices. Ultimately, data analytics in agriculture is crucial in ensuring global food security by empowering farmers to overcome challenges” (*Alagar. 2024, March 13*)

La cita enfatiza el potencial transformador de la analítica de datos en la agricultura y en los tractores, subrayando cómo esta herramienta puede impulsar la innovación, aumentar la productividad y fortalecer la seguridad alimentaria. Destaca cómo los datos a tiempo real permiten a los agricultores tomar decisiones más informadas sobre el uso de recursos y la gestión de cultivos, lo que a su vez contribuye a una agricultura más eficiente y sostenible. Al abordar este cuello de botella en nuestro segundo entregable y al optimizar el rendimiento de nuestras gráficas y los datos CSV, podemos alcanzar un avance significativo en la satisfacción del cliente y, en general, en la creación de un tractor más eficiente.

## B. Antecedentes

En el análisis del código de nuestro segundo entregable, se identificaron varios problemas que contribuyen a un retraso considerable en el despliegue gráfico utilizando Matplotlib en la Raspberry Pi. La función `run_plot()` era responsable tanto de la lectura del puerto serial como de la escritura de datos en un archivo CSV y la actualización de los gráficos. Esta combinación de tareas en una sola función causaba bloqueos y ralentizaba el proceso general de visualización de datos en tiempo real.

A continuación se presenta el código problemático:

Python

```
# Library imports
import numpy as np
import csv
import serial
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# Code imports
import plot_RPi_settings
import plot_serial_settings as set

# List and matrix for live plotting
x_values = list(range(0, set.X_RANGE))
parameters_values = np.zeros((set.X_RANGE, set.NUMBER_OF_VARIABLES))

# Figure and axes declaration
fig, axes = plt.subplots(set.NUMBER_OF_VARIABLES, 1, figsize=set.PLOT_SIZE)
fig.subplots_adjust(hspace=0.5)
scatter_plots = [ax.plot(x_values, parameters_values[:, i], 'g-')[0] for i, ax in
enumerate(axes)]

for i in range(set.NUMBER_OF_VARIABLES):
    axes[i].set_xlabel('Samples')
    axes[i].set_xlim(0, set.X_RANGE)

axes[0].set_ylabel('Throttle')
axes[0].set_ylim(set.THROTTLE_BOUNDS[0], set.THROTTLE_BOUNDS[1])
```

```

axes[1].set_ylabel('Engine Speed')
axes[1].set_ylim(set.ENGINE_SPEED_BOUNDS[0], set.ENGINE_SPEED_BOUNDS[1])

axes[2].set_ylabel('Vehicle Speed')
axes[2].set_ylim(set.VEHICLE_SPEED_BOUNDS[0],
set.VEHICLE_SPEED_BOUNDS[1])

axes[3].set_ylabel('Gear')
axes[3].set_ylim(set.GEAR_BOUNDS[0], set.GEAR_BOUNDS[1])

# Create CSV file
with open(set.CSV_FILE_PATH, mode='w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(['Throttle', 'Engine Speed', 'Vehicle Speed', 'Gear'])

# Instantiate Serial object
ser = serial.Serial(set.SERIAL_PORT, set.BAUDRATE)

def run_plot(frame):
    global parameters_values, ser

    # Read values, decode bytes to string and remove leading/trailing whitespaces
    rx_data = ser.readline().decode('utf_8').strip()

    # Split the received string by commas
    values = rx_data.split(',')

    # Check if all values are received
    if len(values) == 4:
        print("Data received: ", rx_data)

    # Write values to CSV
    with open(set.CSV_FILE_PATH, mode='a', newline='') as file:
        writer = csv.writer(file)
        writer.writerow([float(value) for value in values])

```

```

# Add new values to parameters values matrix
new_row = np.array([float(value) for value in values])
parameters_values = np.vstack([parameters_values, new_row])

# Limit parameters values matrix to set number of items
parameters_values = parameters_values[-set.X_RANGE:, :]

# Update lines with new values
for j, scatter_plot in enumerate(scatter_plots):
    scatter_plot.set_ydata(parameters_values[:, j])
else:
    # Handle the case when all values are not received properly
    print("Incomplete data received: ", rx_data)

return scatter_plots

if __name__ == "__main__":
    ani = FuncAnimation(fig, run_plot, frames=range(set.X_RANGE), blit=True)
    plt.show()

```

En este código, la función `run_plot()` realiza múltiples tareas simultáneamente: lectura de datos desde el puerto serial, escritura de datos en un archivo CSV y actualización de gráficos. Esta combinación de tareas en una única función provoca bloqueos y ralentiza el proceso general, resultando en una visualización gráfica muy lenta y poco eficiente. La implementación de mejoras es necesaria para optimizar el desempeño y lograr una visualización en tiempo real más fluida.

## C. Programa

### 1. Unoptimized code

Python

### # Library imports

```
import numpy as np
```

```
import csv
```

```
import serial
```

```
import matplotlib.pyplot as plt
```

```
from matplotlib.animation import FuncAnimation
```

### # Code imports

```
import plot_RPi_settings
```

```
import plot_serial_settings as set
```

### # List and matrix for live plotting

```
x_values = list(range(0, set.X_RANGE))
```

```
parameters_values = np.zeros((set.X_RANGE, set.NUMBER_OF_VARIABLES))
```

### # Figure and axes declaration

```
fig, axes = plt.subplots(set.NUMBER_OF_VARIABLES, 1, figsize=set.PLOT_SIZE)
```

```
fig.subplots_adjust(hspace=0.5)
```

```
scatter_plots = [ax.plot(x_values, parameters_values[:, i], 'g-')[0] for i, ax in enumerate(axes)]
```

```
for i in range(set.NUMBER_OF_VARIABLES):
```

```
    axes[i].set_xlabel('Samples')
```

```
for i in range(set.NUMBER_OF_VARIABLES):
```

```
    axes[i].set_xlim(0, set.X_RANGE)
```

```
axes[0].set_ylabel('Throttle')
```

```
axes[0].set_ylim(set.THROTTLE_BOUNDS[0] - set.THROTTLE_BOUNDS[0],  
set.THROTTLE_BOUNDS[1] + set.THROTTLE_BOUNDS[0])
```

```
axes[1].set_ylabel('Engine Speed')
```

```
axes[1].set_ylim(set.ENGINE_SPEED_BOUNDS[0] - set.ENGINE_SPEED_BOUNDS[0],
set.ENGINE_SPEED_BOUNDS[1] + set.ENGINE_SPEED_BOUNDS[0])
```

```
axes[2].set_ylabel('Vehicle Speed')
axes[2].set_ylim(set.VEHICLE_SPEED_BOUNDS[0] -
set.VEHICLE_SPEED_BOUNDS[0], set.VEHICLE_SPEED_BOUNDS[1] +
set.VEHICLE_SPEED_BOUNDS[0])
```

```
axes[3].set_ylabel('Gear')
axes[3].set_ylim(set.GEAR_BOUNDS[0] - set.GEAR_BOUNDS[0],
set.GEAR_BOUNDS[1] + set.GEAR_BOUNDS[0])
```

**# Create CSV file**

```
with open(set.CSV_FILE_PATH, mode = 'w', newline = '') as file:
    writer = csv.writer(file)
    writer.writerow(['Throttle', 'Engine Speed', 'Vehicle Speed', 'Gear'])
```

**# Instantiate Serial object**

```
ser = serial.Serial(set.SERIAL_PORT, set.BAUDRATE)
```

```
def run_plot(frame):
```

```
    global parameters_values, ser
```

**# Read values, decode bytes to string and remove leading/trailing whitespaces**

```
rx_data = ser.readline().decode('utf_8').strip()
```

**# Split the received string by commas**

```
values = rx_data.split(',')
```

**# Check if all values are received**

```
if len(values) == 4:
```

```

print("Data received: ", rx_data)

# Write values to CSV

with open(set.CSV_FILE_PATH, mode = 'a', newline = "") as file:
    writer = csv.writer(file)
    writer.writerow([float(value) for value in values])

# Add new values to parameters values matrix

new_row = np.array([float(value) for value in values])
parameters_values = np.vstack([parameters_values, new_row])

# Limit parameters values matrix to set number of items

parameters_values = parameters_values[-set.X_RANGE : , :]

# Update lines with new values

for j, scatter_plot in enumerate(scatter_plots):
    scatter_plot.set_ydata(parameters_values[:, j])

else:

# Handle the case when all values are not received properly

print("Incomplete data received: ", rx_data)

return scatter_plots

if __name__ == "__main__":
    ani = FuncAnimation(fig, run_plot, frames=range(set.X_RANGE), blit=True)
    plt.show()

```

## 2. Optimized code

Python

```
"""
```

Code divided into three threads,  
one reads the serial port,  
the second writes to the CSV file  
and the other plots.

The serial port reading thread  
puts the data into a queue.  
The remaining threads retrieve  
the data instantaneously to perform  
their duty.

Since tasks are simultaneous,  
they can't block each other.

```
"""
```

**# Library imports**

```
import numpy as np
```

```
import csv
```

```
import serial
```

```
import matplotlib.pyplot as plt
```

```
from matplotlib.animation import FuncAnimation
```

**# Code imports**

```
import plot_RPi_settings
```

```
import plot_serial_settings as set
```

**# Added - Threading imports**

```
import threading
```

```
import queue
```

**# Added - Function for receiving data**

```

def receive_data():

    try:

        # Open the serial port

        ser = serial.Serial(set.SERIAL_PORT, set.BAUDRATE)
        print("Serial port opened successfully.")

        while True:

            if ser.in_waiting > 0:

                try:

                    # Read values, decode bytes to string and remove leading/trailing whitespaces

                    rx_data = ser.readline().decode('utf_8').strip()

                    # Split the received string by commas

                    values = rx_data.split(',')

                    # Check if all values are received

                    if len(values) == 4:

                        print("Data received: ", rx_data)
                        data_queue.put(values)

                    else:

                        # Handle the case when all values are not received properly

                        print("Incomplete data received: ", rx_data)

                except UnicodeDecodeError as e:

```

```

        print("Error decoding data:", e)

except serial.SerialException as e:

    print("Error:", e)

finally:

    # Close the serial port

    if ser.is_open:

        ser.close()
        print("Serial port closed.")

# Added - Function for storing in CSV

def store_csv():

    try:

        while True:

            with open(set.CSV_FILE_PATH, mode = 'a', newline = "") as file:

                writer = csv.writer(file)

                if file.tell() == 0:

                    writer.writerow(['Throttle', 'Engine Speed', 'Vehicle Speed', 'Gear'])

            # Write values to CSV

            values = data_queue.get()
            writer.writerow([float(value) for value in values])

```

```

        file.close()

except Exception as e:

    print("Error writing to CSV:", e)

# Added - Function for updating the plot

def update_plot(frame):

    global parameters_values

    try:

        values = data_queue.get()

        # Add new values to parameters values matrix

        new_row = np.array([float(value) for value in values])
        parameters_values = np.vstack([parameters_values, new_row])

        # Limit parameters values matrix to set number of items

        parameters_values = parameters_values[-set.X_RANGE : , :]

        # Update lines with new values

        for j, scatter_plot in enumerate(scatter_plots):
            scatter_plot.set_ydata(parameters_values[:, j])

    except queue.Empty:

        pass

    return scatter_plots

# List and matrix for live plotting

```

```
x_values = list(range(0, set.X_RANGE))
parameters_values = np.zeros((set.X_RANGE, set.NUMBER_OF_VARIABLES))
```

### # Figure and axes declaration

```
fig, axes = plt.subplots(set.NUMBER_OF_VARIABLES, 1, figsize=set.PLOT_SIZE)
fig.subplots_adjust(hspace=0.5)
scatter_plots = [ax.plot(x_values, parameters_values[:, i], 'g-')[0] for i, ax in
enumerate(axes)]
```

```
for i in range(set.NUMBER_OF_VARIABLES):
    axes[i].set_xlabel('Samples')
```

```
for i in range(set.NUMBER_OF_VARIABLES):
    axes[i].set_xlim(0, set.X_RANGE)
```

```
axes[0].set_ylabel('Throttle')
axes[0].set_ylim(set.THROTTLE_BOUNDS[0] - set.THROTTLE_BOUNDS[0],
set.THROTTLE_BOUNDS[1] + set.THROTTLE_BOUNDS[0])
```

```
axes[1].set_ylabel('Engine Speed')
axes[1].set_ylim(set.ENGINE_SPEED_BOUNDS[0] - set.ENGINE_SPEED_BOUNDS[0],
set.ENGINE_SPEED_BOUNDS[1] + set.ENGINE_SPEED_BOUNDS[0])
```

```
axes[2].set_ylabel('Vehicle Speed')
axes[2].set_ylim(set.VEHICLE_SPEED_BOUNDS[0] -
set.VEHICLE_SPEED_BOUNDS[0], set.VEHICLE_SPEED_BOUNDS[1] +
set.VEHICLE_SPEED_BOUNDS[0])
```

```
axes[3].set_ylabel('Gear')
axes[3].set_ylim(set.GEAR_BOUNDS[0] - set.GEAR_BOUNDS[0],
set.GEAR_BOUNDS[1] + set.GEAR_BOUNDS[0])
```

### # Added - Queue for communication between threads

```
data_queue = queue.Queue()
```

```
# Added - Start threads
```

```
receive_thread = threading.Thread(target=receive_data)  
store_csv_thread = threading.Thread(target=store_csv)
```

```
receive_thread.daemon = True  
store_csv_thread.daemon = True
```

```
receive_thread.start()  
store_csv_thread.start()
```

```
if __name__ == "__main__":  
    ani = FuncAnimation(fig, update_plot, interval=50, blit=True, cache_frame_data=False)  
    plt.show()
```

El cambio en nuestro código anterior se centra en la reorganización de las tareas en tres hilos de ejecución separados, lo que permite una ejecución simultánea y evita bloqueos. Uno de los hilos se encarga de leer los datos del puerto, otro de guardarlos en el archivo CSV y el tercero de realizar la graficación de los datos.

Estos hilos se comunican entre sí a través de una cola (queue), donde el hilo que lee el puerto introduce los datos en la cola y los otros dos hilos los recuperan instantáneamente para realizar sus tareas respectivas (guardar en el archivo CSV o graficar). Además, en la función “ani”, se eliminó la parte de “frames” para permitir que “blit” optimice el proceso de dibujado al calcular solo la parte que ha cambiado y necesita ser re-dibujada.

También se agregó el parámetro “interval=50”, reduciendo el tiempo entre frames a 50 milisegundos, lo que hace que la animación se vea más fluida y suave en comparación con el valor predeterminado de 200 milisegundos. Este ajuste en el intervalo mejora la experiencia del usuario al proporcionar una visualización más dinámica y receptiva de los datos.

#### a. Código anterior

```
if __name__ == "__main__":
    ani = FuncAnimation(fig, run_plot, frames=range(set.X_RANGE), blit=True)
    plt.show()
```

*Figura 21. Código anterior*

b. Nuevo código

```
if __name__ == "__main__":
    ani = FuncAnimation(fig, update_plot, interval=50, blit=True, cache_frame_data=False)
    plt.show()
```

*Figura 22. Nuevo código*

D. Video

[https://drive.google.com/file/d/1eJ1NIV3\\_bZDpUw-DB6u2Dky2Idmw5Bv3/view?usp=sharing](https://drive.google.com/file/d/1eJ1NIV3_bZDpUw-DB6u2Dky2Idmw5Bv3/view?usp=sharing)

### Interfaz gráfica desarrollada en Python

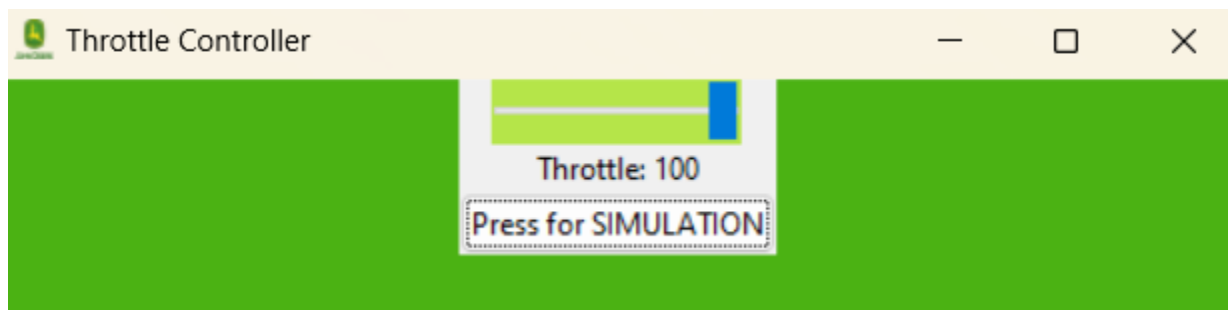
Para el desarrollo de la interfaz gráfica a ejecutar en la Raspberry Pi se seleccionó la librería Tkinter. En esta, se programó un elemento de tipo slider con el que el usuario puede controlar el valor del parámetro de aceleración o Throttle del tractor.

Inicialmente, la comunicación serial y la GUI son activadas. Al presionar las teclas de derecha o izquierda, se obtiene el valor actual del slider, se le resta uno y se actualiza tanto en el objeto como en su etiqueta. Posteriormente, si el botón de simulación ha sido presionado, tal dato de aceleración es enviado por el puerto serial esto con el propósito de ser recolectado por la placa STM32.

Esta última, en una de sus tareas revisa si se encuentra un dato disponible en el registro de datos del periférico USART, lo recupera, y dependiendo de si se presionó el botón de modo

simulación en el teclado matricial, este es usado como entrada del modelo de transmisión automática.

Así pues, un caso de uso es tal que el usuario desea manipular el Throttle para que éste alcance su límite desde la terminal gráfica. Mueve el slider hasta la posición que corresponde al 100%, este número es transmitido de la Raspberry Pi a la placa STM32, el dato se consigue y suponiendo que el modo simulación fue activado anteriormente, este pasa a alimentar el modelo en lugar de aquel proporcionado por el potenciómetro.



*Figura 22. Controlador de Throttle como interfaz de usuario*

## **VII. Conclusiones Individuales**

### Roberto Raymundo Gómez Vargas:

Trabajar en el sistema inteligente de conducción para tractores me ha brindado una comprensión profunda de la complejidad y el potencial de la automatización en la agricultura. Este proyecto no solo me ha permitido desarrollar habilidades técnicas en programación con C/C++ y Python, y en el uso de herramientas como STM32CubeIDE, sino que también me ha enseñado la importancia de una buena comunicación y colaboración en equipo. La interacción entre la Raspberry Pi y la STM32, la optimización del código para mejorar la eficiencia y la creación de un prototipo funcional, han sido retos que me han preparado para futuros proyectos

de ingeniería. A través de esta experiencia, he aprendido a valorar la importancia de la planificación detallada, la implementación precisa y la continua iteración para lograr soluciones efectivas y sostenibles en el ámbito tecnológico.

Juan Manuel Félix Campos:

El proyecto de diseño de un sistema de conducción inteligente para tractores utilizando STM32 y Raspberry Pi ha sido una ventana al futuro de la agricultura y la tecnología. A lo largo de este proyecto, he comprendido cómo los sistemas electrónicos y los microcontroladores pueden transformar la eficiencia y la seguridad en el campo agrícola. Desde la programación de la STM32 para recolectar datos de dispositivos hasta el uso de la Raspberry Pi para procesamiento avanzado y visualización de datos, este proyecto me ha permitido integrar conocimientos de hardware y software de manera práctica. Además, enfrentar problemas como la latencia en la adquisición de datos y la sincronización de tareas me ha enseñado a abordar y resolver problemas técnicos con una mentalidad orientada a la mejora continua y la innovación.

Eugenio Javier Martinez Gastelum:

Trabajar en el proyecto de diseño de un sistema inteligente de conducción para tractores utilizando tarjetas STM32 y Raspberry Pi ha sido una experiencia profundamente enriquecedora que me ha permitido adquirir competencias técnicas y habilidades de colaboración cruciales para mi desarrollo profesional. Desde la conceptualización inicial con visualizaciones gráficas en Python hasta la integración y prototipado mediante comunicación UART, el proyecto me ha enseñado la importancia de la planificación, la comunicación entre dispositivos y la recopilación de datos en tiempo real. Implementar la solución completa, desarrollando software en C/C++ y Python, y creando una interfaz gráfica para manipular parámetros del tractor, me ha brindado una comprensión más profunda de la programación y el control de sistemas. A lo largo de este proceso, he aprendido a enfrentar y resolver problemas de latencia y sincronización, lo que ha

resultado en un sistema más robusto y fiable, consolidando mis habilidades en el desarrollo de sistemas embebidos y el trabajo en equipo bajo la supervisión de expertos.

#### Ivan Rodriguez Nuñez:

Participar en el desarrollo de un sistema inteligente de conducción para tractores con STM32 y Raspberry Pi me ha permitido experimentar de primera mano el ciclo completo de desarrollo de un proyecto de ingeniería. Desde los primeros bocetos y la conceptualización del comportamiento del tractor, pasando por la implementación de comunicación UART y la representación gráfica de los datos, hasta la creación de una interfaz gráfica funcional, cada etapa del proyecto me ha ofrecido desafíos únicos. He aprendido a equilibrar la teoría con la práctica, a optimizar el código para reducir la latencia y a sincronizar tareas críticas, todo mientras colaboraba estrechamente con mis compañeros de equipo. Esta experiencia me ha demostrado la importancia de la interdisciplinariedad y el trabajo en equipo en la resolución de problemas complejos.

### **VIII. Trabajo a Futuro**

#### Pruebas Extensivas:

Realizaremos pruebas en diversos entornos agrícolas para asegurar la robustez y adaptabilidad del sistema. Estas pruebas nos permitirán evaluar el rendimiento del tractor inteligente en diferentes condiciones y escenarios, identificando y resolviendo posibles problemas antes de la implementación final. Al probar en una variedad de entornos, garantizamos que el sistema sea confiable y eficiente, adaptándose a las necesidades específicas de cada operación agrícola y asegurando su éxito en el campo.

#### Expansión de Funcionalidades:

Para aumentar la autonomía y las capacidades del tractor, planeamos integrar más sensores y tecnologías de comunicación avanzadas. Esto permitirá una mejor recopilación de datos en tiempo real y una mayor precisión en las operaciones. Además, desarrollaremos una interfaz de usuario más intuitiva y accesible para los operadores, facilitando la interacción con el

sistema y mejorando la eficiencia y la experiencia de uso. Estas mejoras no solo optimizarán el rendimiento del tractor, sino que también contribuirán a una mayor seguridad y comodidad para los usuarios.

## **IX. Bibliografía**

Alagar. (2024, March 13). Utilizing the power of data analytics in Agriculture. IABAC®.<https://iabac.org/blog/utilizing-the-power-of-data-analytics-in-agriculture#:~:text=Data%20analytics%20not%20only%20aids,harm%2C%20and%20promotes%20sustainable%20practices>.